CARD: MVP capabilities and final-project plan

1. What works now

Updated for scoring model weighted-v3 and the 6 October 2026 presentation. CARD evaluates resources an AI coding agent proposes to use and records a score, explanation and Allow, Needs review or Deny.

Four artifact types: software packages, agent skills/instructions, MCP tool servers and remote URLs. They share one request format and one review workflow.

Evaluation: enter a name and session, then optional source, existence/signature reports, age, downloads, static text and permissions. The engine inspects these supplied inputs locally; it does not fetch or execute the resource.

Scoring: published severity ratios and weighted-average aggregation come from Zahan et al., IEEE Security & Privacy (2023). Package/skill/MCP group weights are 30/20/30/20, totaling 100. Match each group's raw severity, then normalize against all applicable capacity: 250, or 300 for URLs. There is no clipping. Default review is 20 and deny 50. Age/downloads have no penalty. Group ranges and classifications are documented adaptations awaiting validation.

Session context: prior non-allowed requests of other artifact types can influence the current evaluation. A shared name strengthens that context cue. The latest 20 earlier events in the same session are considered. That window is an implementation limit.

Interactive controls: Overview totals, a session dropdown and trace, a review inbox, a searchable request list, type and decision filters, session filters, pagination, and a detailed evidence inspector.

Review and policy: a human may resolve held requests with Allow/Deny and a note. The original score and receipt are preserved. Thresholds and correlation can be saved; the preview slider creates no evaluation.

Replays: four synthetic workflows use the local API. Known publisher: 0/Allow; Young integration: 20/Review; Linked trust signals: URL 50/Deny then skill 30/Review; correlation off gives skill 10/Allow. Over-privileged unknown source: 50/Deny, from weighted 30 + 20.

Persistence and audit: evaluations, policy and reviews are saved locally. Export JSON and verify the original hash-linked evaluation chain. New evaluations include the model version and policy used; older results retain their original scoring and hashes.

Interface: desktop/mobile layout, keyboard navigation, reconnect and bundled Manrope/Public Sans fonts. The inspector includes paper references, group weights, raw severities, the full weighted calculation and supporting signals already covered.

2. How to run and demonstrate it

Install Node.js 22 or newer. In the CARD folder, run the following once. Initial dependency installation needs internet.

```
npm ci
npm run build
npm start
```

Open http://127.0.0.1:4318/. Keep the terminal running. After setup, npm start is sufficient unless source changes require rebuilding. Development uses npm run dev with the interface on port 4317.

Before presenting: set review 20, deny 50 and correlation on. Old records retain earlier calculations; replay fresh examples.

Suggested sequence: show Overview; replay Known publisher; replay Linked trust signals and inspect the 10 + 20 = 30 skill result; turn correlation off, save and replay to show 10/Allow; restore correlation; submit the review example below; resolve it with a note; export the audit.

New evaluation example: select MCP server. Name: presentation-notes. Session: presentation-review-01, using a new ID for each rehearsal. Source: https://example.invalid/tools. Existence: Unknown. Signature: Reported unsigned. Age: 6. Downloads: 18. Static content: "Summarise the selected project notes." Check only network:egress.

Expected result: raw provenance 25 and capability 25, out of full capacity 250. Contributions are 25/250 x 100 = 10 each. **10 + 10 = 20, Needs review.** Age/downloads add nothing; duplicate provenance is already covered. A note enables Allow/Deny.

Functional validation commands: npm run check and npm run test:browser. Browser tests use isolated temporary stores, installed Edge on Windows or Playwright Chromium elsewhere. They verify functionality, not empirical detection accuracy.

Audit verification: download the JSON export and run the following with its actual location.

```
npm run verify:audit -- "C:/path/to/card-audit.json"
```

Research explanation

The numeric scale is described in a peer-reviewed IEEE publication, but the complete CARD model is an adaptation. Security conference research motivates hallucination, typosquatting, content and indirect-injection checks. No claim of measured detection accuracy is made. Read CARD-Scoring-Rationale.pdf for each category's reasoning and complete references.

Main reference: N. Zahan et al., "OpenSSF Scorecard: On the Path Toward Ecosystem-Wide Automated Security Metrics," IEEE Security & Privacy, 21(6), 76-88, 2023. DOI: 10.1109/MSEC.2023.3279773. Accessible manuscript: https://arxiv.org/html/2208.03412v3#S2.

3. What the final project will add

Current limits

Interception: the MVP returns decisions through an API. It does not automatically intercept shell commands, skill reads, MCP traffic or browser requests. A cooperating caller must submit a request before use and obey the verdict.

Evidence: existence, signature, age, downloads and permissions are caller reports. There are no live registry, DNS, domain reputation or signature-verification checks.

Detection: the package catalog and static text patterns are small. There is no semantic injection classifier, complete package analysis, execution sandbox or consensus hallucinated-name database. Legitimate inputs can match; malicious inputs can be missed.

Operations: this is a single-user local prototype. It has no authentication, reviewer identity, roles, tenants or multi-process database coordination.

Audit: receipts are SHA-256 hash-linked, not digitally signed or externally anchored. They can reveal changed original evaluations or broken links, but cannot authenticate their author or detect a fully rewritten chain. Review/policy events are separate and not authenticated by that chain.

Research validation: no labelled 200-500-case corpus, measured false-positive rate, comparative detection rate or end-to-end agent latency benchmark has been completed.

Planned completion stages

Connect and enforce: build supported agent adapters, package/skill hooks, an MCP proxy and controlled HTTP egress. Completion requires denied operations to stop and held operations to pause/resume correctly.

Verify evidence: add independent registries, signed provenance and appropriate reputation checks. Distinguish verified results from unavailable or uncertain information.

Expand analysis: broaden name/content detectors and investigate a trained scoring model only with reliable labelled evidence. Review the current severity assignments with domain experts.

Build and evaluate a benchmark: assemble realistic malicious and benign workflows. Compare against relevant baselines, measure per-type detection and false positives, run correlation on/off ablation, vary thresholds and report latency. Use separate calibration and test data and report negative results honestly.

Harden operation: add persistent database coordination, identities/permissions for reviewers, authenticated audit receipts and secure deployment controls.

The final contribution is a tested enforcement system plus a reproducible research evaluation. The current MVP demonstrates the interface, explanatory scoring flow, correlation, review and audit mechanisms needed to reach it.